Министерство науки и высшего образования Российской Федерации
ФГАОУ ВО «Уральский федеральный университет
имени первого Президента России Б.Н. Ельцина»
Институт новых материалов и технологий
Кафедра «Теплофизика и информатика в металлургии»

Автоматизация сборки проекта с помощью Jenkins

ОТЧЕТ
по практической работе № 4
по дисциплине «Основы методологии Development Operation»

Направление 09.03.02 «Информационные системы и технологии» (уровень бакалавриата)
Образовательная программа
09.03.02/33.02 «Информационные системы и технологии» (СУОС)

Студент
группы НМТ-433901 О.М. Малышева

Преподаватель:
профессор, д.т.н. В.В. Лавров

Екатеринбург
2026

СОДЕРЖАНИЕ

Практическая работа «Автоматизация сборки проекта с помощью Jenkins» 3
1.1 Цель работы 3
1.2 Ход проведения работы 3
1.2.1 Разработка тестового приложения Visual Studio .NET Core 3
1.2.2 Создание контейнера с использованием Docker 4
1.2.3 Создание файла docker-compose.yml 4
1.2.4 Создание файла Jenkinsfile 4
1.2.5 Размещение приложения в системе контроля версий GitHub 4
1.2.6 Настройка автоматической сборки проекта в Jenkins и интеграции с GitHub 5
1.2.7 Выполнение автоматической сборки проекта в Jenkins и анализ результатов 6
1.2.8 Демонстрация автоматической сборки и доставки проекта при корректировке функциональности веб-приложения 7
1.3 Выводы 8
Приложение А Листинг программного кода приложения «Калькулятор» (контроллер) 9
Приложение Б Листинг программного кода приложения «Калькулятор» (appsettings.json) 10
Приложение В Листинг программного кода приложения «Калькулятор» (appsettings.Development.json) 11
Приложение Г Листинг программного кода приложения «Калькулятор» (Program.cs) 12
Приложение Д Листинг файла Dockerfile 13
Приложение Е Листинг файла docker-compose.yml 14
Приложение Ж Листинг файла Jenkinsfile 15

Практическая работа
«Автоматизация сборки проекта с помощью Jenkins»

1.1 Цель работы

Цель практической работы – разработать веб-приложение, разместить его исходный код в системе контроля версий GitHub и настроить автоматизированную сборку и развертывание приложения с помощью Jenkins и Docker. В результате изменения, отправленные в удалённый репозиторий, должны использоваться Jenkins для получения исходного кода, сборки Docker-образа и обновления работающего контейнера.

1.2 Ход проведения работы

1.2.1 Разработка тестового приложения Visual Studio .NET Core

В среде Visual Studio было разработано веб-приложение «Калькулятор» на платформе ASP.NET Core MVC. Приложение предоставляет пользователю веб-форму для ввода двух чисел и выбора арифметической операции. Реализованы операции сложения, вычитания, умножения и деления. При попытке деления на ноль приложение выводит сообщение об ошибке. Основная логика обработки запроса размещена в контроллере CalculatorController, а представление Index.cshtml отвечает за отображение интерфейса.

Для запуска приложения используется конфигурация Kestrel с HTTP-адресом 0.0.0.0:5007. Данный порт выбран для размещения приложения на кафедральном сервере.

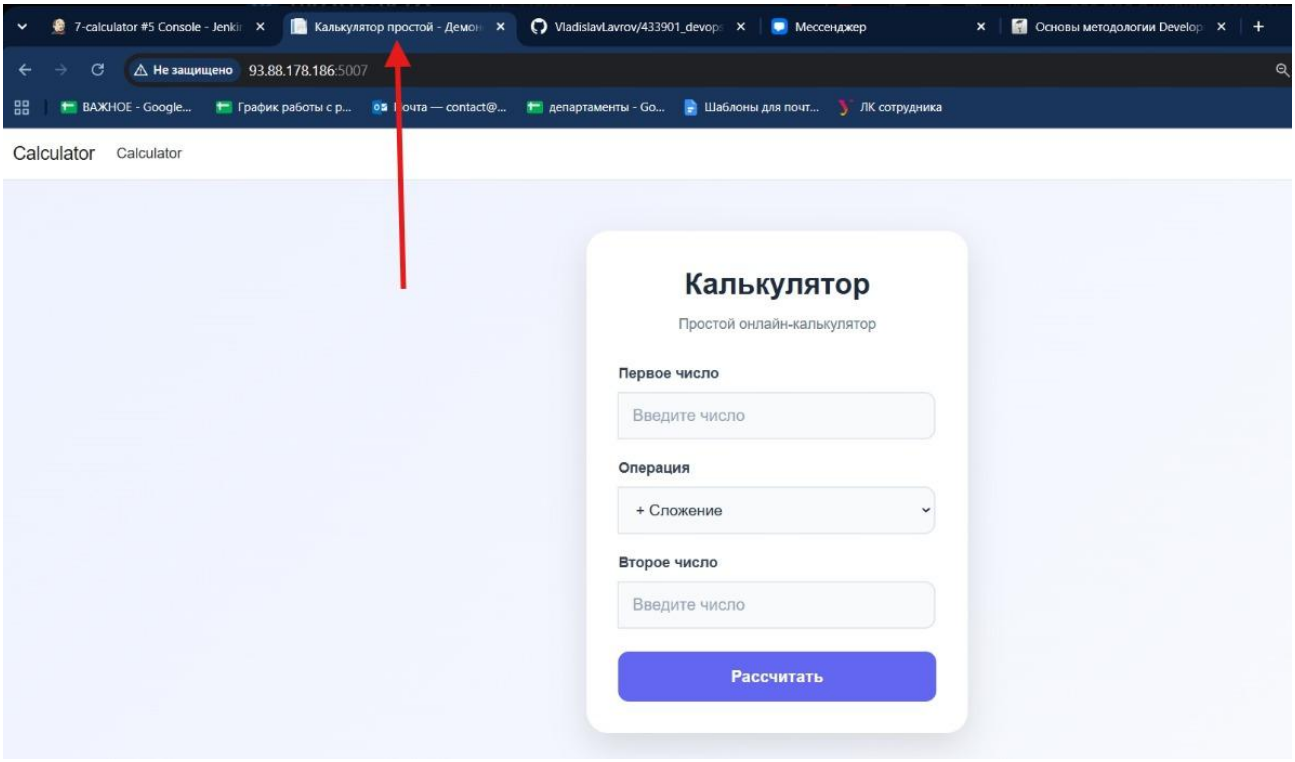


Рисунок 1 – Интерфейс веб-приложения «Калькулятор»

1.2.2 Создание контейнера с использованием Docker

Для упаковки приложения был подготовлен Dockerfile с многоэтапной сборкой. На этапе build используется образ .NET SDK 9.0: выполняется восстановление зависимостей и сборка проекта в конфигурации Release. На этапе publish приложение публикуется в каталог /app/publish. На финальном этапе используется образ ASP.NET Runtime 9.0, в который копируются опубликованные файлы. Контейнер запускает приложение командой dotnet 7_Calculator.dll и предоставляет порт 5007.

Такой подход позволяет отделить среду сборки от среды выполнения и запускать приложение в контейнере без установки SDK на целевой машине.

1.2.3 Создание файла docker-compose.yml

Для упрощения управления контейнеризацией проекта был создан файл docker-compose.yml, в котором задаются параметры запуска веб-приложения в Docker. В конфигурации указываются параметры сервиса и публикации портов. Для доступа к калькулятору используется порт 5007.

Файл размещён в корневом каталоге проекта 7_Calculator. Его использование позволяет хранить настройки контейнеризации отдельно от исходного кода приложения и упрощает повторный запуск контейнера с заданными параметрами.

1.2.4 Создание файла Jenkinsfile

В корне репозитория размещён Jenkinsfile с декларативным pipeline. В нём определены три этапа: Checkout, Build Docker image и Deploy Calculator. На этапе Checkout выполняется получение исходного кода из SCM. На этапе Build Docker image создаётся Docker-образ с тегом 7_calculator-web:latest. На этапе Deploy Calculator удаляется предыдущий контейнер calculator_olesya_5007 (если он существует) и запускается новый контейнер с публикацией порта 5007.

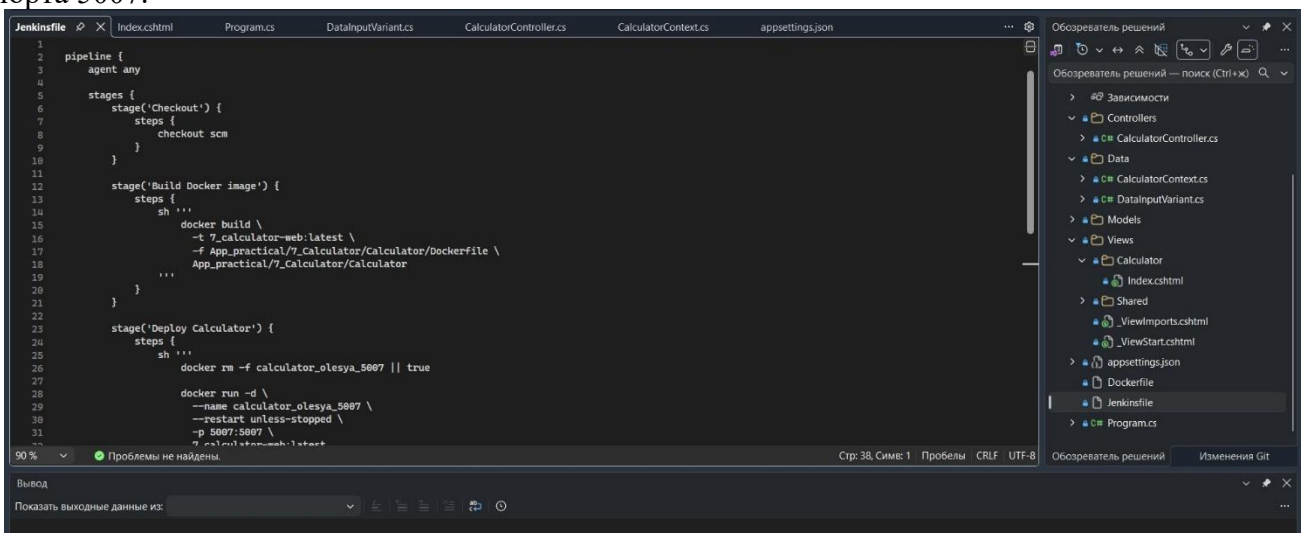


Рисунок 2 – Jenkinsfile проекта в Visual Studio

1.2.5 Размещение приложения в системе контроля версий GitHub

Исходный код проекта размещён в удалённом репозитории GitHub: https://github.com/VladislavLavrov/433901_devops_MalyshevaOlesya. Для фиксации изменений использовался GitHub Desktop. После изменения представления калькулятора был создан

коммит «изменение 2», содержащий изменение файла Index.cshtml, и выполнена отправка коммита в ветку master.

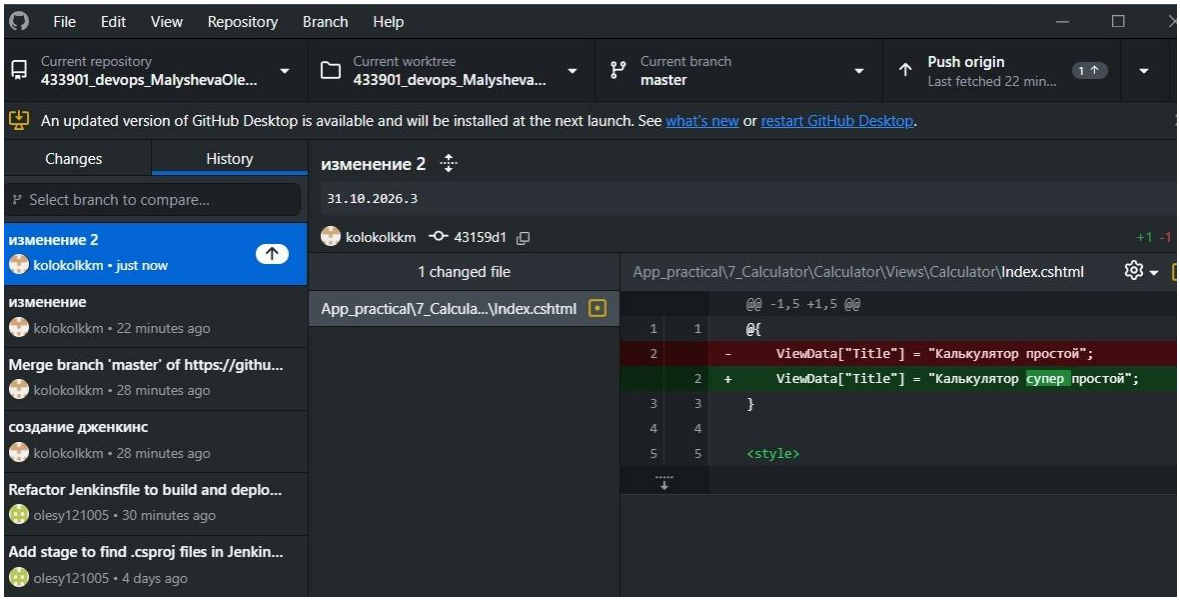


Рисунок 3 – Коммит с изменением веб-интерфейса в GitHub Desktop

1.2.6 Настройка автоматической сборки проекта в Jenkins и интеграции с GitHub

В Jenkins создана задача типа Pipeline с именем 7-calculator. В общих настройках указана ссылка на GitHub-проект. В разделе Pipeline выбран способ определения сценария из SCM, в качестве системы контроля версий указан Git, задан URL репозитория, а в качестве Branch Specifier – */master. Таким образом, Jenkins получает Jenkinsfile и исходный код из удалённой ветки master.

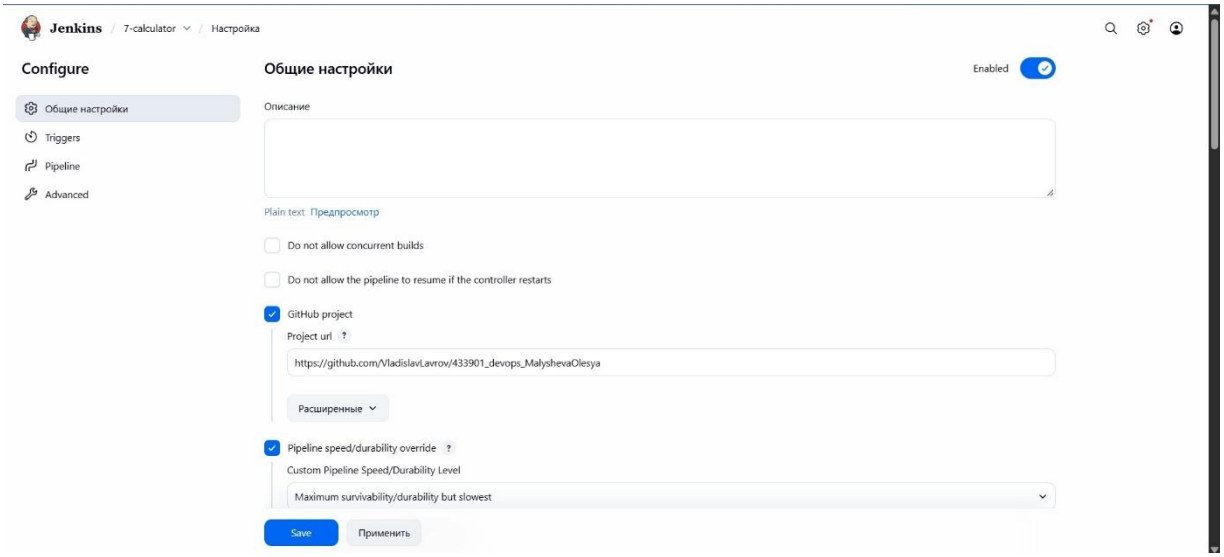


Рисунок 4 – Создание задачи

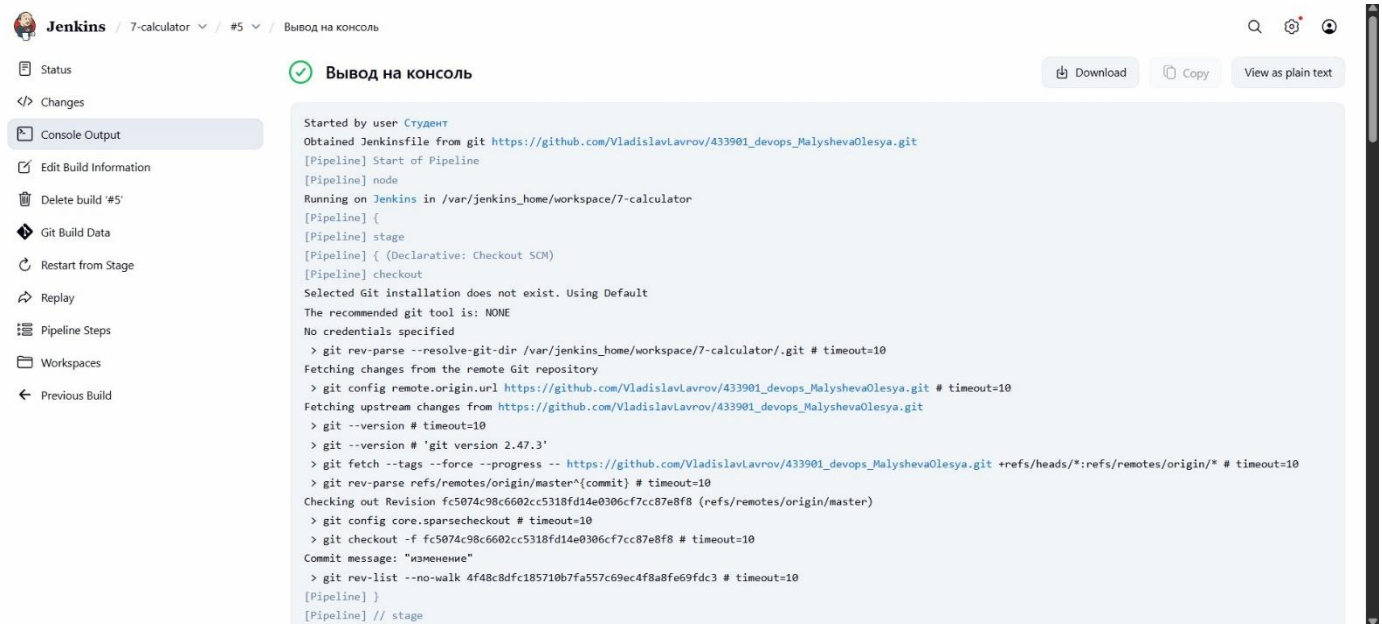
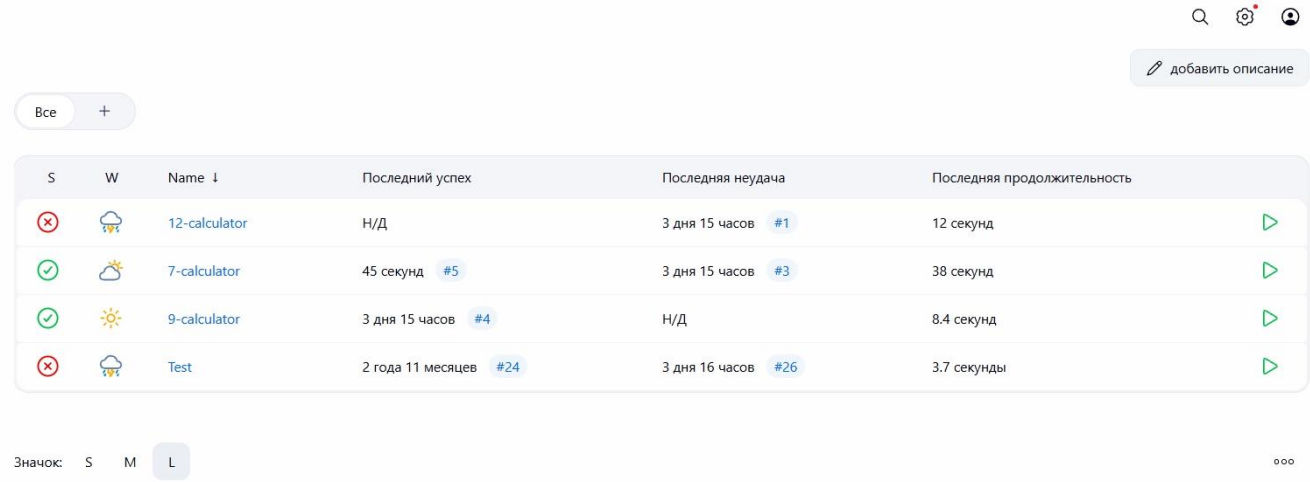


Рисунок 5 – Настройка Pipeline и подключение GitHub

1.2.7 Выполнение автоматической сборки проекта в Jenkins и анализ результатов

После отправки изменений в репозиторий была запущена задача 7-calculator. В консольном выводе сборки № 5 отображается получение Jenkinsfile из GitHub, загрузка изменений из ветки master и выполнение pipeline на агенте Jenkins. Сборка завершилась с успешным статусом. На главной странице Jenkins для задачи 7-calculator также отображается успешная последняя сборка № 5.



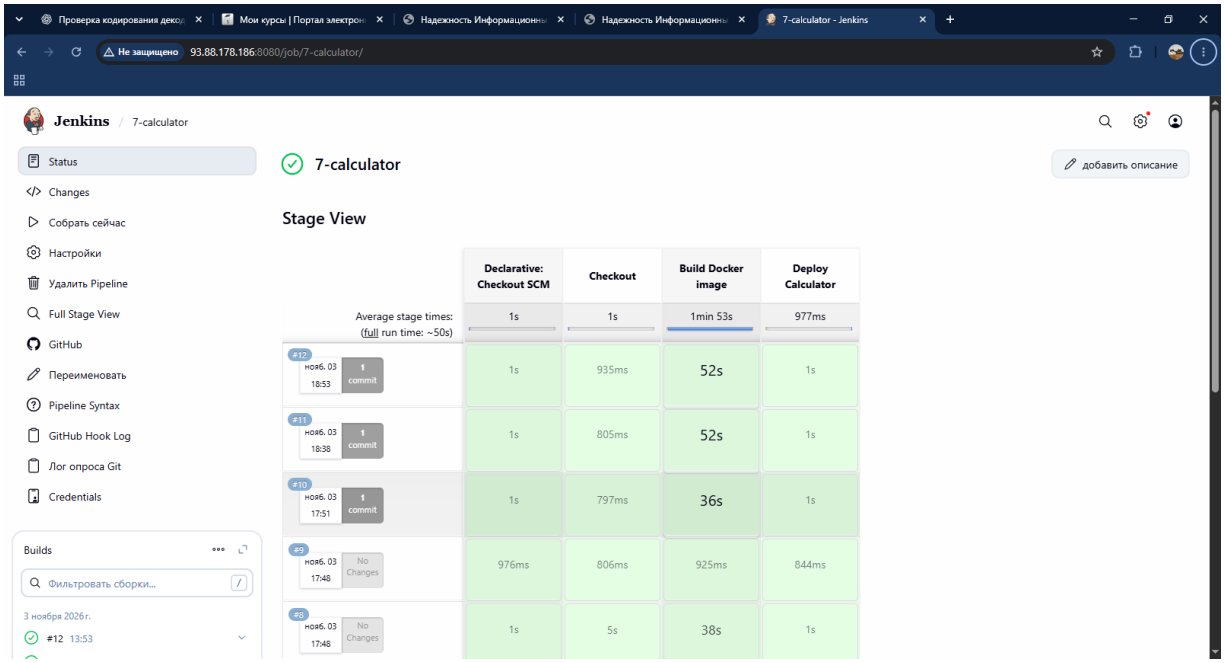


Рисунок 6 – Консольный вывод успешной сборки Jenkins № 5

1.2.8 Демонстрация автоматической сборки и доставки проекта при корректировке функциональности веб-приложения

Для проверки доставки был изменена надпись в представлении Index.cshtml: вместо исходного текста «Калькулятор простой» указано «Калькулятор супер простой». Изменение сохранено в Git, зафиксировано коммитом и отправлено в удалённый репозиторий. После выполнения pipeline Jenkins обновлённая версия приложения стала доступна на кафедральном сервере по адресу http://93.88.178.186:5007.

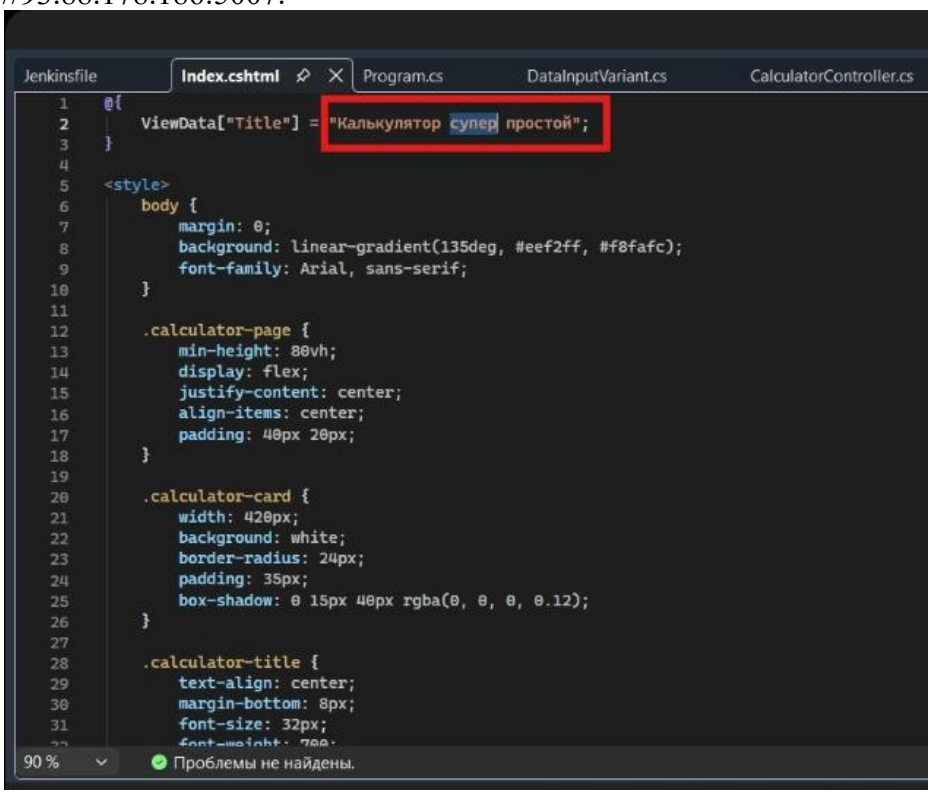


Рисунок 7 – Изменение текста в представлении Index.cshtml

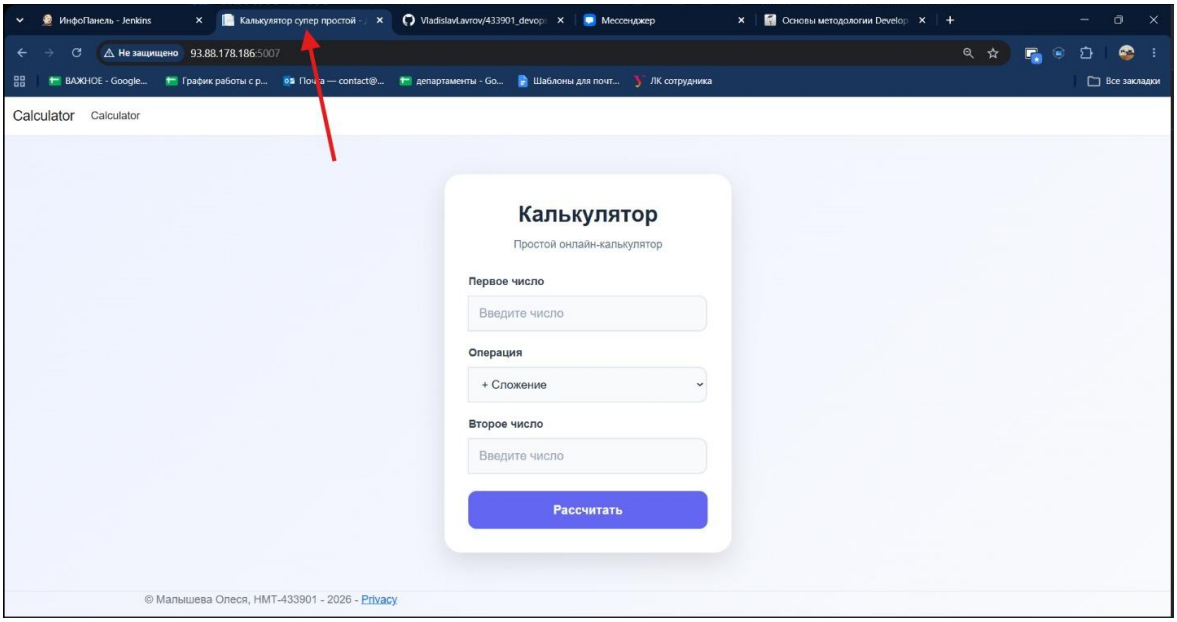


Рисунок 8 – Обновлённая версия калькулятора на сервере

1.3 Выводы

В ходе практической работы было разработано веб-приложение «Калькулятор» на ASP.NET Core MVC, подготовлен Dockerfile для контейнеризации, а исходный код размещён в GitHub. В Jenkins создана Pipeline-задача 7-calculator, которая получает исходный код из ветки master, собирает Docker-образ и развёртывает контейнер приложения на порту 5007. Успешное выполнение сборки № 5 подтверждено консольным выводом Jenkins. Корректировка текста веб-интерфейса и отображение обновлённой версии на сервере продемонстрировали работоспособность процесса сборки и доставки изменений.

Приложение А Листинг программного кода приложения «Калькулятор» (CalculatorController.cs)

```
using Microsoft.AspNetCore.Mvc;

namespace Calculator.Controllers
{
    public enum Operation
    {
        Add,
        Subtract,
        Multiply,
        Divide
    }

    public class CalculatorController : Controller
    {
        [HttpGet]
        public IActionResult Index()
        {
            return View();
        }

        [HttpPost]
        [ValidateAntiForgeryToken]
        public IActionResult Calculate(double num1, double num2, Operation operation)
        {
            double result = 0;

            switch (operation)
            {
                case Operation.Add:
                    result = num1 + num2;
                    break;

                case Operation.Subtract:
                    result = num1 - num2;
                    break;

                case Operation.Multiply:
                    result = num1 * num2;
                    break;

                case Operation.Divide:
                    if (num2 == 0)
                    {
                        ModelState.AddModelError("", "На ноль делить нельзя.");
                        return View("Index");
                    }

                    result = num1 / num2;
                    break;
            }

            ViewBag.Result = result;

            return View("Index");
        }
    }
}
```

Приложение Б Листинг программного кода приложения «Калькулятор» (appsettings.json)

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=mariadb; Database=myWebCalculatorDb; Uid=root; Pwd=***;
Character Set=utf8; ConvertZeroDatetime=True;"
  },
  "Kestrel": {
    "Endpoints": {
      "Http": {
        "Url": "http://0.0.0.0:5007" // Заменить последние 2 цифры порта на порядковый но-
мер из студенческого журнала. Например, порт 5012 соответствует номеру 12
      }
    }
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft": "Warning",
      "Microsoft.Hosting.Lifetime": "Information"
    }
  },
  "AllowedHosts": "*"
}
```

Приложение В Листинг программного кода приложения «Калькулятор» (appsettings.Development.json)

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=93.88.178.186; Port=5023; Database=myWebCalculatorDb;
Uid=root; Pwd=***; Character Set=utf8; ConvertZeroDatetime=True;"
  },
  "DetailedErrors": true,
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft": "Warning",
      "Microsoft.Hosting.Lifetime": "Information"
    }
  }
}
```

Приложение Г Листинг программного кода приложения «Калькулятор» (Program.cs)

```
using Microsoft.EntityFrameworkCore;
using Calculator.Data;

var builder = WebApplication.CreateBuilder(args);

// Добавление сервисов
builder.Services.AddControllersWithViews();

string mariadbCS = builder.Configuration.GetConnectionString("DefaultConnection");

builder.Services.AddDbContext<CalculatorContext>(options =>
{
    options.UseMySql(
        mariadbCS,
        new MySqlServerVersion(new Version(10, 5, 15))
    );
});

builder.Services.AddRazorPages();


var app = builder.Build();

// Настройка HTTP-конвейера
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Calculator/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.UseAuthorization();


app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Calculator}/{action=Index}/{id?}");

app.Run();
```

Приложение Д Листинг файла Dockerfile

```
# Используем базовый образ ASP.NET 10.0
FROM mcr.microsoft.com/dotnet/aspnet:9.0 AS base

# Рабочая директория внутри контейнера
WORKDIR /app

# Используем .NET 10 SDK для сборки
FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build

# Копируем файлы проекта в контейнер
COPY . /src

# Рабочая директория для сборки
WORKDIR /src

# Восстанавливаем зависимости
RUN dotnet restore

# Собираем приложение
RUN dotnet build "./7_Calculator.csproj" -c Release -o /app/build

# Публикуем приложение
FROM build AS publish
RUN dotnet publish "./7_Calculator.csproj" -c Release -o /app/publish

# Финальный образ
FROM base AS final
WORKDIR /app

# Копируем опубликованное приложение
COPY --from=publish /app/publish ./

# Порт приложения внутри контейнера
EXPOSE 5007

# Запускаем приложение
ENTRYPOINT ["dotnet", "7_Calculator.dll"]
```

Приложение Е Листинг файла docker-compose.yml

```
version: '3.7'

services:
  mariadb:
    container_name: web-mariadb
    hostname: mariadb
    image: mariadb:10.5
    restart: always
    environment:
      MARIADB_ROOT_PASSWORD: password
      MARIADB_DATABASE: webdb
      MARIADB_USER: webdb
      MARIADB_PASSWORD: password
    ports:
      - "0.0.0.0:5023:3306"

  web:
    container_name: web-app-calculator
    hostname: app
    build: Calculator/
    ports:
      - "0.0.0.0:5015:5015"
```

Приложение Ж Листинг файла Jenkinsfile

```
pipeline { agent any
stages {
    stage('Checkout') {
        steps {
            checkout scm
        }
    }

    stage('Build Docker image') {
        steps {
            sh '''
                docker build \
                  -t 7_calculator-web:latest \
                  -f App_practical/7_Calculator/Calculator/Dockerfile \
                  App_practical/7_Calculator/Calculator
            '''
        }
    }

    stage('Deploy Calculator') {
        steps {
            sh '''
                docker rm -f calculator_olesya_5007 || true

                docker run -d \
                  --name calculator_olesya_5007 \
                  --restart unless-stopped \
                  -p 5007:5007 \
                  7_calculator-web:latest
            '''
        }
    }
}
}
```